

Assignment 01: Linux Kernel Compilation & Parallelism Benchmarking

Student Name: Muhammad Saad (23L-2620 / BDS-6A)

Kernel Version Used: Linux 6.18.7-saad-linux

Environment: Ubuntu (VMware Virtual Machine)

1. System Specifications

The experiments were performed inside a VMware virtual machine running Ubuntu Linux. The host system provided sufficient resources for kernel compilation.

CPU: Intel Core i5 (11th Generation)

Cores Allocated to VM: 4 cores

Memory Allocated: 6 GB RAM

Storage Allocated: 70 GB (expanded during compilation)

A screenshot of the `lscpu` command output is attached with this report as required.

```
saad@DELL:~$ lscpu
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Address sizes:                45 bits physical, 48 bits virtual
Byte Order:                  Little Endian
CPU(s):                       4
On-line CPU(s) list:         0-3
Vendor ID:                   GenuineIntel
Model name:                   11th Gen Intel(R) Core(TM) i5-1145G7 @ 2.60GHz
CPU family:                   6
Model:                       140
Thread(s) per core:          1
Core(s) per socket:          4
Socket(s):                    1
Stepping:                     1
BogoMIPS:                     2995.20
Flags:                        fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pg
                              e mca cmov pat pse36 clflush mmx fxsr sse sse2 ss h
                              t syscall nx pdpe1gb rdtscp lm constant_tsc arch_pe
                              rfmon rep_good nopl xtopology tsc_reliable nonstop_
                              tsc cpuid tsc_known_freq pni pclmulqdq ssse3 fma cx
                              16 pcid sse4_1 sse4_2 x2apic movbe popcnt aes xsave
                              avx f16c rdrand hypervisor lahf_lm abm 3dnowprefet
                              ch pti ssbd ibrs ibpb stibp fsgsbase tsc_adjust bmi
                              1 avx2 smep bmi2 erms invpcid avx512f avx512dq rdse
                              ed adx snap avx512ifma clflushopt clwb avx512cd sha
                              _ni avx512bw avx512vl xsaveopt xsavec xgetbv1 xsave
                              s arat avx512vbmi unip avx512_vbmi2 gfni vaes vpclm
                              ulqdq avx512_vnni avx512_bitalg avx512_vpopercntdq rd
                              pid movdiri movdir64b fsrm avx512_vp2intersect md_c
                              lear flush_l1d arch_capabilities

Virtualization features:
Hypervisor vendor:           VMware
Virtualization type:         full
Caches (sum of all):
L1d:                          192 KiB (4 instances)
L1i:                          128 KiB (4 instances)
L2:                            5 MiB (4 instances)
L3:                            8 MiB (1 instance)
NUMA:
NUMA node(s):                 1
NUMA node0 CPU(s):           0-3
Vulnerabilities:
Gather data sampling:         Unknown: Dependent on hypervisor status
Ghostwrite:                   Not affected
Indirect target selection:    Mitigation; Aligned branch/return thunks
Itlb multihit:               Not affected
L1tf:                         Mitigation; PTE Inversion
Mds:                          Mitigation; Clear CPU buffers; SMT Host state unkno
                              wn
Meltdown:                     Mitigation; PTI
Mmio stale data:              Not affected
Old microcode:                Not affected
Reg file data sampling:       Not affected
Retbleed:                     Mitigation; IBRS
Spec rstack overflow:         Not affected
Spec store bypass:            Mitigation; Speculative Store Bypass disabled via p
                              rctl
Spectre v1:                   Mitigation; usercopy/swapgs barriers and __user poi
                              nter sanitization
Spectre v2:                   Mitigation; IBRS; IBPB conditional; STIBP disabled;
                              RSB filling; PBRSE-eIBRS Not affected; BHI SW loop
                              , KVM SW loop
Srbds:                        Not affected
Tsa:                          Not affected
Tsx async abort:              Not affected
Vnscape:                      Not affected

saad@DELL:~$
```

2. Kernel Customization

2.1 Kernel Source

The latest stable Linux kernel was downloaded from `kernel.org`:

- **Kernel Version:** 6.18.7

2.2 Configuration Change

The existing system configuration was copied, and the kernel configuration menu was opened. The timer frequency was verified to be set to **1000 Hz** under. This configuration improves desktop responsiveness.

2.3 Kernel Signature Modification

To embed a custom signature into the kernel, the top-level Makefile was edited. The `EXTRAVERSION` field was modified as follows:

```
EXTRAVERSION = -saad-linux
```

This ensures the custom kernel can be uniquely identified.

3. Experiment Results

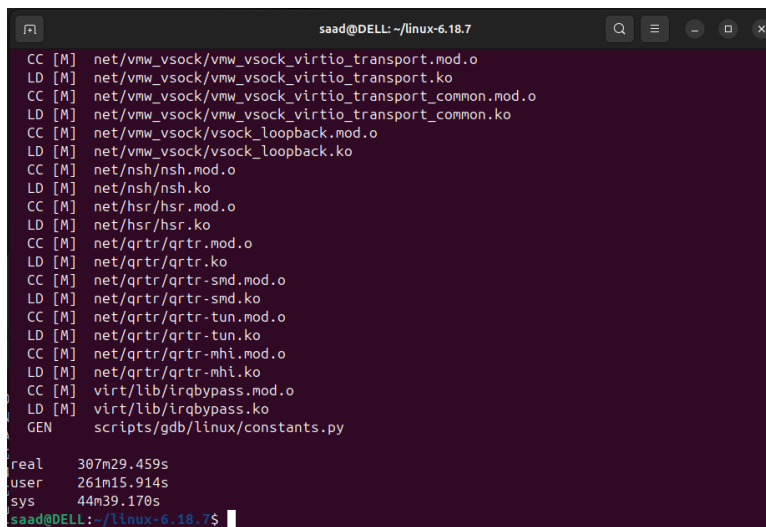
Experiment A: Single-Core Compilation

The kernel was compiled using a single core.

Result:

- **Real Time:** ~307 minutes ($\approx$ 5 hours 7 minutes)

This run was completed successfully and served as the baseline for comparison.



```
saad@DELL: ~/linux-6.18.7
CC [M] net/vmw_vsock/vmw_vsock_virtio_transport.mod.o
LD [M] net/vmw_vsock/vmw_vsock_virtio_transport.ko
CC [M] net/vmw_vsock/vmw_vsock_virtio_transport_common.mod.o
LD [M] net/vmw_vsock/vmw_vsock_virtio_transport_common.ko
CC [M] net/vmw_vsock/vsock_loopback.mod.o
LD [M] net/vmw_vsock/vsock_loopback.ko
CC [M] net/nsh/nsh.mod.o
LD [M] net/nsh/nsh.ko
CC [M] net/hsr/hsr.mod.o
LD [M] net/hsr/hsr.ko
CC [M] net/qrtr/qrtr.mod.o
LD [M] net/qrtr/qrtr.ko
CC [M] net/qrtr/qrtr-smc.mod.o
LD [M] net/qrtr/qrtr-smc.ko
CC [M] net/qrtr/qrtr-tun.mod.o
LD [M] net/qrtr/qrtr-tun.ko
CC [M] net/qrtr/qrtr-mhi.mod.o
LD [M] net/qrtr/qrtr-mhi.ko
CC [M] virt/lib/irqbypass.mod.o
LD [M] virt/lib/irqbypass.ko
GEN scripts/gdb/linux/constants.py

real    307m29.459s
user    261m15.914s
sys     44m39.170s
saad@DELL: ~/linux-6.18.7$
```

Experiment B: Multi-Core Compilation

After cleaning the build artifacts, the kernel was compiled using multiple cores.

Result:

- **Real Time:** ~150 minutes ($\approx$ 2 hours 30 minutes)

This demonstrates a significant reduction in compilation time when parallelism is utilized.

```
saad@DELL: ~/linux-6.18.7
LD [M] net/ife/ife.ko
LD [M] net/openvswitch/openvswitch.ko
LD [M] net/openvswitch/vport-vxlan.ko
LD [M] net/openvswitch/vport-geneve.ko
LD [M] net/openvswitch/vport-gre.ko
LD [M] net/vmw_vsock/vsock.ko
LD [M] net/vmw_vsock/vsock_diag.ko
LD [M] net/vmw_vsock/vmw_vsock_vmci_transport.ko
LD [M] net/vmw_vsock/vmw_vsock_virtio_transport.ko
LD [M] net/vmw_vsock/vmw_vsock_virtio_transport_common.ko
LD [M] net/vmw_vsock/vsock_loopback.ko
LD [M] net/nsh/nsh.ko
LD [M] net/hsr/hsr.ko
LD [M] net/qrtr/qrtr.ko
LD [M] net/qrtr/qrtr-smd.ko
LD [M] net/qrtr/qrtr-tun.ko
LD [M] net/qrtr/qrtr-mhi.ko
LD [M] virt/lib/irqbypass.ko

real    150m49.097s
user    491m50.047s
sys     77m34.548s
saad@DELL:~/linux-6.18.7$
```

4. Speedup Analysis

The speedup was calculated using the formula:

$$\text{Speedup} = T_{\text{serial}} / T_{\text{parallel}}$$

Substituting the measured values:

$$\text{Speedup} = 307 / 150 \approx 2.04\times$$

Discussion

Although 4 CPU cores were used, the speedup is not linear (i.e., not 4× faster). This is expected due to several factors:

- Some parts of the kernel build process are inherently serial.
- Disk I/O becomes a bottleneck during compilation.
- Virtualization overhead from VMware limits perfect parallel scaling.
- According to **Amdahl's Law**, the presence of serial components limits maximum achievable speedup.

The observed speedup is realistic and consistent with theoretical expectations.

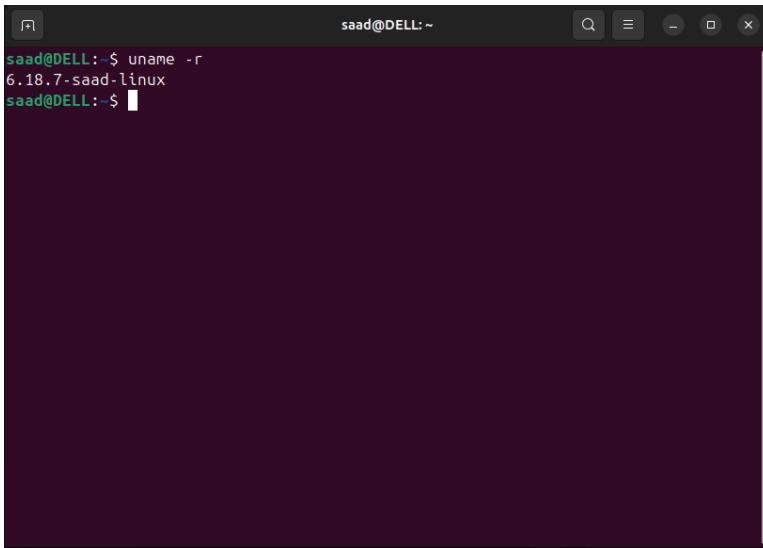
5. Installation and Verification

After successful compilation, the kernel and modules were installed. The system was rebooted, and the newly compiled kernel was selected from the GRUB boot menu after that verification was performed.

Output:

6.18.7-saad-linux

A screenshot of this output is included as proof of successful installation.

A terminal window with a dark background and light green text. The window title is 'saad@DELL: ~'. The command 'uname -r' has been executed, and the output is '6.18.7-saad-linux'. The prompt 'saad@DELL:~\$' is visible on the next line.

```
saad@DELL:~$ uname -r
6.18.7-saad-linux
saad@DELL:~$
```

6. Conclusion

This assignment successfully demonstrated the process of compiling, customizing, and installing a Linux kernel. The performance comparison between single-core and multi-core compilation clearly illustrates the benefits and limitations of parallelism. The experiment reinforces theoretical concepts from parallel computing and provides practical experience with real-world system-level software.